

Frontend Overview

The Complete Modern Web Development Landscape

■ LEARNING OUTCOME

By the end of this module you will understand the complete modern frontend stack, how browsers render web pages, the difference between frameworks, and have a clear learning roadmap from HTML to deployable React apps.

■ Skills You Will Gain

- Understand how browsers request and render web pages
- Distinguish HTML, CSS, and JavaScript responsibilities
- Know the modern frontend toolchain from editor to deployment
- Compare React, Vue, Angular and when to choose each
- Understand SPAs vs MPAs vs SSR vs SSG
- Read and navigate the Node.js / npm ecosystem

■ Frontend engineers are the highest-demand web developers in 2024. React alone appears in 60% of frontend job postings. Understanding the full stack from browser rendering to deployment separates seniors from juniors.

SECTION 1

How Browsers Work

The Critical Rendering Path -- What Happens When You Type a URL

1. DNS Lookup: Domain name -> IP address (e.g. example.com -> 93.184.216.34) 2. TCP/TLS: Establish encrypted connection to server 3. HTTP Request: Browser sends GET request for the HTML file 4. HTML Parsing: Browser reads HTML, builds DOM (Document Object Model) 5. CSS Parsing: Browser reads CSS, builds CSSOM (CSS Object Model) 6. Render Tree: DOM + CSSOM combined into renderable tree 7. Layout: Calculate position and size of every element 8. Paint: Draw pixels to screen 9. Composite: Layer GPU-accelerated elements JS blocks rendering! Script tags without async/defer pause HTML parsing. Use: script defer src=app.js to load JS without blocking.

SECTION 2

The Three Layers of the Web

Layer	Technology	Responsibility	Example
Structure	HTML5	Content and semantic meaning	h1, nav, article, form
Presentation	CSS3	Visual appearance and layout	colors, fonts, flexbox, grid
Behaviour	JavaScript	Interactivity and dynamic updates	events, DOM changes, API calls

```
<!-- HTML: structure --> <button class='btn' id='like-btn'>Like</button> /* CSS: presentation */
.btn { padding:10px 20px; background:#7C3AED; color:white; border-radius:6px; border:none;
cursor:pointer; } .btn:hover { opacity:0.9; } .btn.liked { background:#EC4899; } // JS: behaviour
const btn = document.getElementById('like-btn'); let liked = false; btn.addEventListener('click',
() => { liked = !liked; btn.classList.toggle('liked', liked); btn.textContent = liked ? 'Liked!' :
'Like'; });
```

SECTION 3

The Modern Frontend Toolchain

1	Editor VS Code with ESLint, Prettier, GitLens, and language support extensions installed
2	Node.js and npm Runtime for tooling. npm manages packages. Use nvm to manage Node versions
3	Package Manager npm, yarn, or pnpm to install and manage project dependencies
4	Bundler/Build Tool Vite (modern, fast) or Webpack bundles JS, CSS, images into optimised output

5	Framework React, Vue, or Svelte for building component-based UIs
6	Type Safety TypeScript adds static types to JavaScript -- catches errors before runtime
7	Linting ESLint catches code problems. Prettier enforces consistent formatting automatically
8	Version Control Git + GitHub. Feature branches, pull requests, code review workflow
9	Testing Vitest and Jest for unit tests. React Testing Library for component tests. Playwright for E2E
10	Deployment Vercel or Netlify: push to GitHub -> automatic preview + production deploy

SECTION 4

Framework Comparison

Framework	Philosophy	Learning Curve	Job Market	Best For
React	Library -- you choose everything else	Medium	Highest -- 60%+ jobs	SPAs, dashboards, complex UIs
Vue.js	Progressive framework -- gentle learning	Low	Medium	Beginners, teams from jQuery
Angular	Full framework -- opinionated, batteries in	High	Enterprise-focused	Large enterprise apps
Svelte	Compile-time framework -- no virtual DOM	Low	Growing	Performance-critical apps
Next.js	React + SSR/SSG + routing + API routes	Medium	Very high	Full-stack React production apps

SPA vs SSR vs SSG -- Three Rendering Strategies

SPA (Single Page Application): Browser downloads one HTML shell + JS bundle. JS renders everything. Pros: fast navigation, rich interactions. Cons: slow initial load, poor SEO. SSR (Server-Side Rendering): Server renders HTML for each request. Client hydrates. Pros: fast first load, good SEO. Cons: server load, complexity. Tools: Next.js, Remix. SSG (Static Site Generation): HTML generated at BUILD time. Serve static files from CDN. Pros: fastest possible, cheapest hosting. Use for blogs, docs, marketing. Tools: Astro, Next.js.

■ PRACTICE Q & A

Q1. What is the DOM and how is it different from HTML?

A: HTML is the source text file. The DOM (Document Object Model) is the browser's in-memory tree -- a live object that JavaScript can read and modify. Changes to the DOM update the page without reloading.

Q2. What does defer on a script tag do?

A: defer tells the browser to download the script in parallel while HTML parses, then execute it AFTER HTML parsing completes. This prevents JS from blocking rendering -- the page appears faster.

Q3. What is the difference between a library and a framework?

A: Library (React): you call its functions. You control the flow. Framework (Angular): it calls your code -- inverts control. Frameworks are more opinionated and provide more structure but less flexibility.

Q4. What is hydration in SSR?

A: After SSR, the browser has static HTML. Hydration is where the JS framework attaches event listeners and state to existing HTML, making it interactive. Before hydration the page looks right but is not interactive.

Q5. What is the critical rendering path?

A: Sequence: HTML parsing -> DOM -> CSS -> CSSOM -> Render Tree -> Layout -> Paint. Optimise with async loading, CSS above fold, and minimal blocking JavaScript.

■ Frontend Overview Cheat Sheet	
DOM	Browser's live JS-accessible page tree
Critical Rendering Path	HTML->DOM->CSSOM->Layout->Paint
script defer	Load JS without blocking render
SPA	JS renders everything -- no page reload
SSR	Server generates HTML per request
SSG	HTML generated at build time -- CDN
React	UI library -- most frontend jobs
Next.js	React + SSR/SSG + routing + API
npm install package	Install dependency
Vite	Modern fast dev server and bundler
ESLint + Prettier	Linting + auto-formatting
Vercel / Netlify	Git push -> auto deploy